

下面我直接给你一份 **LoveAdvisor V2 正式实施方案文档（按天推进版）**。
这份方案不是从零空想，而是基于你们 **V1 已经真实打通的链路** 来做重构：V1 已完成 Coze 工作流、前后端分离、真实 API 调用、S1-S6 链路打通，但也暴露出性能、S2 保守、S4 不稳、S6 冗余等系统性问题。

LoveAdvisor V2 正式实施方案

规则增强型关系决策系统重构

一、V2 项目定义

1.1 项目阶段定义

V1 的目标是：

把项目打通，验证整条链路可运行

V2 的目标是：

把项目做稳、做准、做成可迭代的系统

也就是说，V2 不再是“继续补 prompt”，而是：

- 重构 workflow 职责
- •
重构节点边界
- •
重构模型与规则分工
-

-

重构输出与测试机制

-

1.2 V2 核心升级方向

V1 的关键问题已经非常明确：

-

模型直接承担高层关系判断，稳定性不足

-

-

S2 一旦保守，后续链路全部失真

-

-

S4 既耗时又不稳

-

-

S6 二次推理属于冗余

-

-

系统缺少规则引擎和自动评测机制。

-

因此 V2 的核心原则定义为：

模型负责提信号

规则系统负责下结论

生成层负责把结论变成人话

二、V2 总体架构设计

2.1 V2 推荐架构

用户输入

- S1 文本清洗
 - S2 基础互动信号提取
 - S3 高级关系信号提取
 - R1 后端规则引擎 (relationship_stage / interest_level)
 - S5 策略建议生成
 - O1 输出组装层
 - 前端展示
-

2.2 为什么这样设计

V1 的原始方案是：

- S1 清洗
- S2 提取
- S3 趋势
- S4 关系判断
- S5 建议
- S6 输出

这个方案在 V1 阶段是合理的，因为它帮助你快速完成了 MVP 打通。实施方案和 Day1–Day6 日志都验证了这条链路的可行性。

但 V2 必须升级为下面的职责分配：

模块	V1 角色	V2 角色
S1	清洗	保留，继续做压缩清洗
S2	提取 + 弱判断	只做基础互动信号提取
S3	趋势判断	改为高级关系信号提取
S4	模型判阶段	删除
R1	无	新增后端规则引擎
S5	建议生成	保留，但输入改为“结构化信号 + 规则结果”
S6	模型封装	删除推理属性，改为 O1 组装层

2.3 V2 的高效原则

原则一：高层标签不再直接交给模型

模型不再直接输出：

-

初识期

-

-

暧昧期

-

-

拉扯期

-

-

冷淡期

-

模型只提更低层、更可观测的信号。

原则二：节点必须单一职责

每个节点只做一件事，避免：

-

提取 + 判断混在一起

-

-

判断 + 生成混在一起

-

-

推理 + 封装混在一起

-

原则三：后端引入显式规则

关系阶段和兴趣等级改为代码规则，可控、可测、可复盘。

原则四：输出层不再调用模型

Day 4 和 Day 6 已经证明，输出层应是整合层，不应再承担推理。

三、V2 节点详细设计

3.1 S1：文本清洗节点

职责

-

统一说话人格式

-

-

保留时间顺序

-

-

去除系统噪音

-

-

保留低信息情绪内容

-
-

压缩无效文本长度

-

输入

-

chat_text

-

输出

-

cleaned_chat

-

输出格式

A: ...
B: ...
A: ...

节点配置建议

-

模型：豆包通用 lite

-
-

类型：LLM 节点

-
-

输出长度：尽量控制在原始文本 40%~60%

-
-

不允许输出 reasoning

-
-

不允许 JSON，直接输出标准化文本

-

Prompt 目标

S1 不是摘要器，而是“关系分析预处理器”。

3.2 S2：基础互动信号提取节点

职责

只提取基础互动行为信号，不做关系阶段判断。

输入

- `cleaned_chat`
-

输出 JSON

```
{  
  "initiative": "",  
  "response_length": "",  
  "emotional_tone": "",  
  "topic_depth": "",  
}
```

```
"interaction_reciprocity": "",
"key_signals": []
}
```

字段定义

字段	可选值
initiative	A 更主动 / B 更主动 / 双方接近 / 无法判断
response_length	长 / 中 / 短 / 无法判断
emotional_tone	热 / 温 / 冷 / 无法判断
topic_depth	浅 / 中 / 深 / 无法判断
interaction_reciprocity	正向承接 / 弱承接 / 明显回避 / 无法判断

节点配置建议

- 模型：豆包 1.5 Pro 32K
-
- 类型：LLM 节点
-
- 严格 JSON 输出
-
- key_signals 限制 2-4 条
-

为什么这样设计

Day 1 和 Day 6 已经说明，S2 是上游数据源，必须稳定，但不能再承担过多高层判断。

3.3 S3：高级关系信号提取节点

职责

识别“关系相关特征”，但不直接输出 `relationship_stage`。

输入

- `S2_output`
-
- 必要时可加 `cleaned_chat`
-

输出 JSON

```
{
  "has_intimacy_signal": false,
  "has_relationship_probe": false,
  "has_positive_reciprocity": false,
  "has_rejection_signal": false,
  "has_push_pull_pattern": false,
  "has_sustained_coldness": false,
  "signal_summary": []
}
```

字段说明

字段	含义
<code>has_intimacy_signal</code>	是否有“想你、想见你、来陪我”等亲密拉近信号
<code>has_relationship_probe</code>	是否有“你会想我吗”“你是不是冷了”等关系试探
<code>has_positive_reciprocity</code>	是否有顺着关系话题靠近的正向承接
<code>has_rejection_signal</code>	是否有拒聊、否定、后退、明显回避

字段	含义
has_push_pull_pattern	是否同时存在推进与回避
has_sustained_coldness	是否存在持续低投入，而非单次简短

节点配置建议

-

模型：DeepSeek V3

-
-

类型：LLM 节点

-
-

严格 JSON 输出

-
-

signal_summary 2-3 条即可

-

为什么 S3 要新设计

因为测试样本已经表明，模型难以稳定判“暧昧期”，但识别“是否有亲密信号”“是否有试探”难度要低得多。测试集 A/B/C/D/E 正好适合围绕这些信号来做验证。

3.4 R1：后端规则引擎

职责

根据 S2 + S3 的结构化信号，稳定判定：

-

relationship_stage

-
-

interest_level

-

输入

-

S2_output

-
-

S3_output

-

输出 JSON

```
{  
  "relationship_stage": "",  
  "interest_level": ""  
}
```

判定逻辑建议

relationship_stage

1. 暧昧期

满足任一：

-

has_intimacy_signal = true

-
-

has_relationship_probe = true 且 has_positive_reciprocity = true

-

且:

-

has_rejection_signal = false

-

2. 拉扯期

满足任一:

-

has_push_pull_pattern = true

-

-

has_intimacy_signal = true 且 has_rejection_signal = true

-

3. 冷淡期

满足任一:

-

has_sustained_coldness = true

-

-

has_rejection_signal = true 且 S2 的 emotional_tone = 冷

-

4. 初识期

满足:

-

有基本互动结构

-
-

无强亲密

-
-

无持续冷淡

-
-

无拉扯

-

5. 无法判断
满足：

-

信号极少

-
-

互动结构不足

-

interest_level 判定逻辑

条件	interest_level
双向亲近 + 正向承接明显	高
一方推进，对方未拒绝	中
礼貌回应、浅层互动	低
回避、拒聊、持续冷淡	低
信息极少	无法判断

技术实现建议

-

放在后端 Python 中

-

-

写成单独函数 `infer_relationship_stage(signals)` 和 `infer_interest_level(signals)`

-

-

所有规则写成可修改表，而不是散在业务代码里

-

为什么必须后端规则化

这一步是 V2 的核心升级。

因为你已经验证出：直接让模型判阶段，效果高度不稳定；但规则引擎可以显式控制边界，尤其适合“标准初识/暧昧/冷淡/无法判断”这类测试集回归。

3.5 S5：策略建议生成节点

职责

根据：

-

S2 基础信号

-

-

S3 高级信号

-

-

R1 规则结果

-
-

用户问题

-

生成:

-

心理分析

-
-

风险点

-
-

建议

-
-

下一步动作

-

输入

-

S2_output

-
-

S3_output

-
-

R1_output

-
-

user_question

-

输出 JSON

```
{  
  "psychological_analysis": "",  
  "risk_points": [],  
  "suggestions": [],  
  "next_step": ""  
}
```

节点配置建议

-

模型：豆包 1.5 Pro 32K

-
-

类型：LLM 节点

-
-

suggestions 固定 3 条

-
-

risk_points 2–3 条

-
-

next_step 1 条

-

为什么保留 S5

Day 3 已经证明，用户真正买单的是“我该怎么办”，而不是“你判断得多聪明”。S5 是产品价值层，必须保留。

3.6 01：输出组装层

职责

只做整合，不做推理。

输入

-

S2_output

-
-

S3_output

-
-

R1_output

-
-

S5_output

-

输出 JSON

{

```
"relationship_stage": "",
"interest_level": "",
"key_signals": [],
"signal_summary": [],
"psychological_analysis": "",
"risk_points": [],
"suggestions": [],
"next_step": ""
}
```

实现方式

-

不用 LLM

-
-

后端 Python 直接组装

-
-

缺失值兜底:

-

-

字符串字段 → "无法判断"

- -

数组字段 → []

-

为什么这么做

这就是 Day 4 和 Day 6 已经验证过的正确做法。

四、V2 每一天的实施安排

下面按 7 天重构周期 给你正式排期。

Day 1：V2 架构重整与协议设计

目标

明确 V2 的整体架构和数据协议。

当天任务

1.

画出 V2 新工作流图

2.

3.

删除原架构中的“模型版 S4”

4.

5.

删除原 S6 的推理设计

6.

7.

定义 S2、S3、R1、S5、O1 的输入输出协议

8.

9.

确定前后端数据流

10.

当日产出

-

V2 架构图

-
-

节点职责表

-
-

JSON 协议文档

-
-

后端规则引擎接口定义

-

验收标准

-

所有节点职责不重叠

-
-

输出字段命名统一

-
-

后端能明确知道每一步要接什么数据

-

Day 2: S2 重构

目标

把 S2 从“半判断节点”改造成“基础互动信号提取器”。

当天任务

1.

重写 S2 Prompt

2.

3.

删除 S2 中所有高层关系判断倾向

4.

5.

增加 `interaction_reciprocity`

6.

7.

限制 `key_signals` 数量

8.

9.

用基础/低信息样本测试稳定性

10.

推荐测试

•

A1 初识

•

•

E1 单句

•

•

E2 极短对话

-
-

E3 信息不足

-

当日产出

-

新版 S2 Prompt

-
-

S2 测试记录表

-

验收标准

-

不再大面积“全无法判断”

-
-

能稳定输出基础互动信号

-
-

不直接输出“暧昧/拉扯/冷淡”倾向

-

Day 3: S3 新建

目标

新增高级关系信号识别层。

当天任务

1.

新建 S3 节点

2.

3.

编写 S3 Prompt

4.

5.

测试亲密、试探、回避、拉扯等信号识别

6.

7.

输出严格 JSON

8.

推荐测试

•

B1 标准暧昧

•

•

B2 轻度暧昧

•

•

C1 典型拉扯

-
-

D1 标准冷淡

-

当日产出

-

新版 S3 Prompt

-
-

高级信号测试结果

-

验收标准

-

B 类样本能触发 intimacy / probe

-
-

C 类样本能触发 push_pull

-
-

D 类样本能触发 rejection / coldness

-

Day 4: R1 后端规则引擎实现

目标

正式落地 `relationship_stage / interest_level` 规则判定。

当天任务

1.

在 `backend` 中新增规则模块

2.

3.

编写 `infer_relationship_stage()`

4.

5.

编写 `infer_interest_level()`

6.

7.

将 `S2 + S3` 接入规则引擎

8.

9.

输出标准 `R1_output`

10.

当日产出

-

后端规则函数

-

-

规则表

-

-

示例输入输出

-

验收标准

-

不依赖模型就能给出阶段与兴趣等级

-

-

逻辑可读、可改、可测

-

-

A/B/C/D/E 样本能够跑通第一轮

-

Day 5: S5 重接入

目标

让 S5 从“依赖模型版 S4”切换为“依赖规则结果”。

当天任务

- 1.

修改 S5 Prompt 输入

- 2.

- 3.

让 S5 读取 R1_output

- 4.

5.

优化建议风格，保留自然度

6.

7.

重点测试冷淡与暧昧场景的建议质量

8.

当日产出

-

新版 S5 Prompt

-

-

S5 建议样本输出对比表

-

验收标准

-

心理分析、建议明显更贴合阶段结果

-

-

不再出现“上游一偏，下游全偏”的连锁问题

-

-

文风自然，不明显模板化

-

Day 6: 01 输出层与前后端联调

目标

完成 v2 的整合与展示链路。

当天任务

1.

删除旧 S6 推理节点

2.

3.

在后端实现 O1 组装逻辑

4.

5.

补全缺失字段兜底

6.

7.

修改前端字段适配

8.

9.

跑全链路联调

10.

当日产出

•

标准最终 JSON

•

•

前端联调结果

-
-

错误日志样例

-

验收标准

-

最终输出统一

-
-

不再出现 `s5_output: "{...}"` 这种嵌套字符串

-
-

前端可直接渲染

-

Day 7: V2 全量测试与评估

目标

建立 V2 的正式回归测试。

当天任务

1.

跑完整 A/B/C/D/E 测试集

2.

3.

记录实际输出与预期差异

4.

5.

分类错误类型：

6.

○

过度保守

○

○

误判冷淡

○

○

误判初识

○

○

建议偏推进

○

7.

计算阶段判断通过率

8.

9.

形成 V2 第一版评估报告

10.

当日产出

-

测试记录表

-
-

错误分类表

-
-

V2 阶段性总结

-

验收标准

-

A1 / B1 / D1 必须通过

-
-

E 类不可判样本不乱判

-
-

全链路稳定性明显高于 V1

-

五、每个节点推荐配置汇总

节点	类型	推荐模型/实现	职责
S1	LLM	豆包通用 lite	清洗压缩
S2	LLM	豆包 1.5 Pro 32K	基础互动信号
S3	LLM	DeepSeek V3	高级关系信号
R1	Python 规则	后端代码	关系阶段/兴趣等级

节点	类型	推荐模型/实现	职责
S5	LLM	豆包 1.5 Pro 32K	建议生成
O1	Python 组装	后端代码	输出整合

六、V2 的高效架构要点

6.1 Coze 里只保留必要 LLM 节点

不要再把“能用代码做的事情”继续塞给模型。

6.2 规则尽量后端化

所有关键边界判断统一放后端，便于版本管理和测试。

6.3 输出层绝不再推理

O1 只能组装，不能思考。

6.4 所有节点必须有固定 JSON 协议

这是后续做测试系统和历史记录的基础。

6.5 测试必须围绕样本集

你已有的 A/B/C/D/E 测试集，是 V2 最重要的回归资产。

七、V2 的预期收益

维度	V1	V2 预期
准确率	易受模型波动影响	明显更稳
耗时	S4/S6 拖慢全链路	更短
调试方式	改 prompt 为主	改规则 + 改信号
可解释性	依赖模型输出	规则清晰可追踪
产品化能力	基本可用	可迭代、可测试、可展示

八、最终阶段定义

V2 的阶段名称建议正式写成：

LoveAdvisor V2：规则增强型关系决策系统重构期

你这一步判断是对的：

V1 的价值不是“做到完美”，而是完成了打通验证；**V2** 的任务才是真正把它做成系统。

下一步我可以继续直接给你两样最落地的东西中的一个：

V2 节点 **Prompt** 全套草案，或者 **R1** 后端规则引擎伪代码/正式代码框架。